


```python
# Appendix A

def generate_dispatch_data():
    original = list(range(100 + SEED_DIGIT, 100 + SEED_DIGIT + 12))
    pivot = ID_SUM % len(original)
    rotated = original[pivot:] + original[:pivot]
    target = original[(pivot + 4) % len(original)]
    return original, rotated, target


def linear_search_record(data, target):
    comparisons = 0
    for index, value in enumerate(data):
        comparisons += 1
        if value == target:
            return index, comparisons
    return -1, comparisons


def rotated_binary_search(data, target):
    low, high = 0, len(data) - 1
    comparisons = 0

    while low <= high:
        mid = (low + high) // 2
        comparisons += 1

        if data[mid] == target:
```

Python

Exercises 1, 2, and 3.ipynb

Appendices > Exercises 1, 2, and 3.ipynb > LAST_NAME = "Damoluan"

Jupyter Notebook

Generate | Code | Markdown | Run All | Clear All Outputs | Outline | ... | Select Kernel

```
            if data[mid] == target:
                return mid, comparisons

            if data[low] <= data[mid]:
                if data[low] <= target < data[mid]:
                    high = mid - 1
                else:
                    low = mid + 1
            else:
                if data[mid] < target <= data[high]:
                    low = mid + 1
                else:
                    high = mid - 1

        return -1, comparisons


    original_records, rotated_records, target_record = generate_dispatch_data()
    linear_index, linear_comparisons = linear_search_record(rotated_records, target_record)
    rotated_index, rotated_comparisons = rotated_binary_search(rotated_records, target_record)

    print("Exercise 1: Emergency Dispatch Record Search")
    print("Original records:", original_records)
    print("Rotated records:", rotated_records)
    print("Target record:", target_record)
    print("Linear search index:", linear_index, "| comparisons:", linear_comparisons)
    print("Rotated binary search index:", rotated_index, "| comparisons:", rotated_comparisons)
    print()
```

[11] Python

Exercises 1, 2, and 3.ipynb

Appendices > Exercises 1, 2, and 3.ipynb > LAST_NAME = "Damoluan"

Jupyter Notebook

Generate | Code | Markdown | Run All | Clear All Outputs | Outline | ... | Select Kerne

```
Exercise 1: Emergency Dispatch Record Search
Original records: [108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119]
Rotated records: [109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 108]
Target record: 113
Linear search index: 4 | comparisons: 5
Rotated binary search index: 4 | comparisons: 4
```

```
    # Appendix B

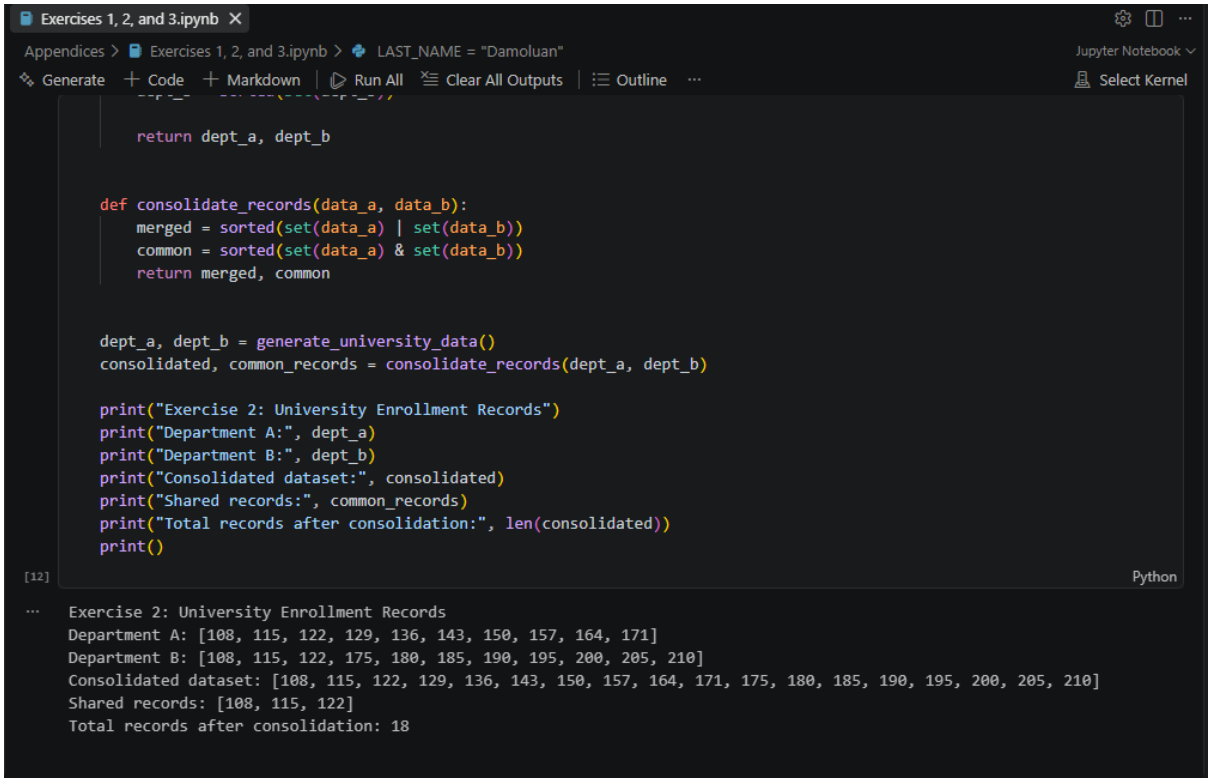
    def generate_university_data():
        dept_a = [100 + SEED_DIGIT + i * 7 for i in range(10)]
        dept_b = [150 + ID_SUM + i * 5 for i in range(8)]

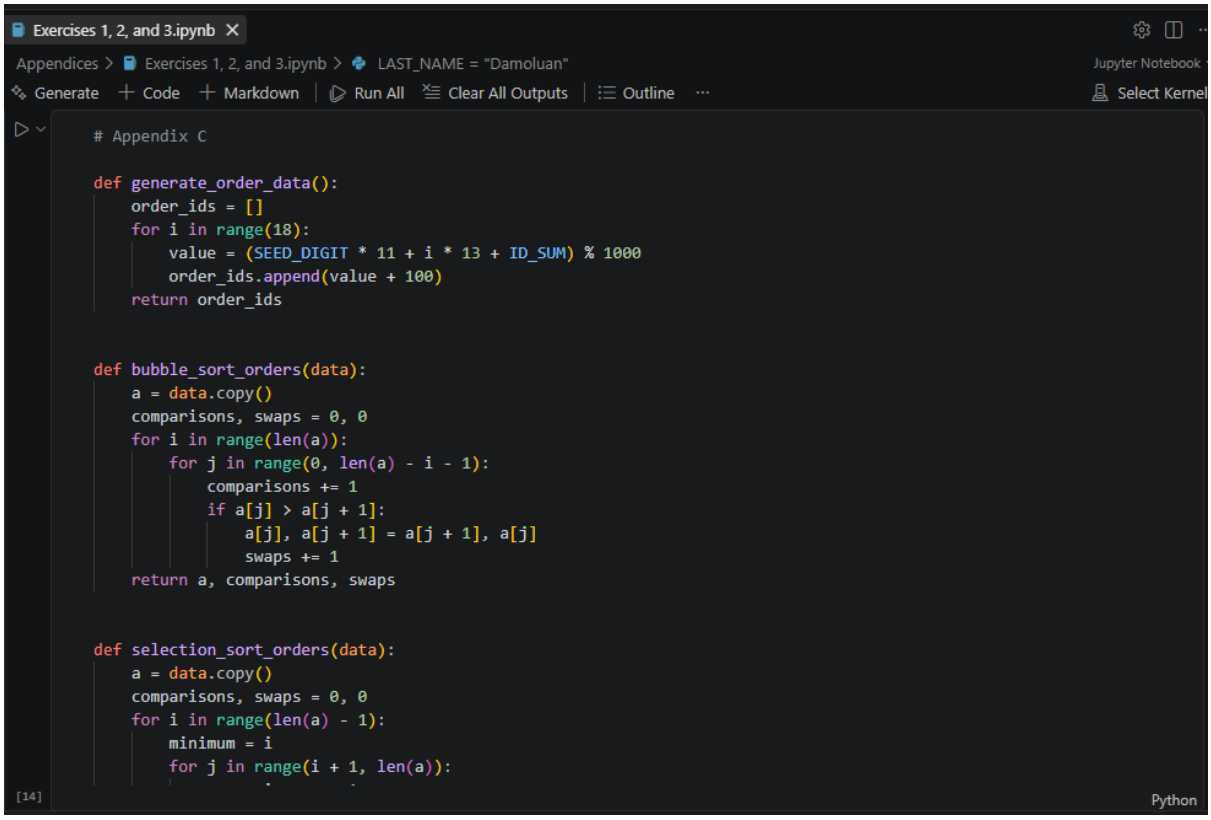
        # Make the records overlap and keep them sorted for matching later.
        dept_a = sorted(set(dept_a))
        dept_b = sorted(set(dept_b))
        for value in dept_a[:3]:
            if value not in dept_b:
                dept_b.append(value)
        dept_b = sorted(set(dept_b))

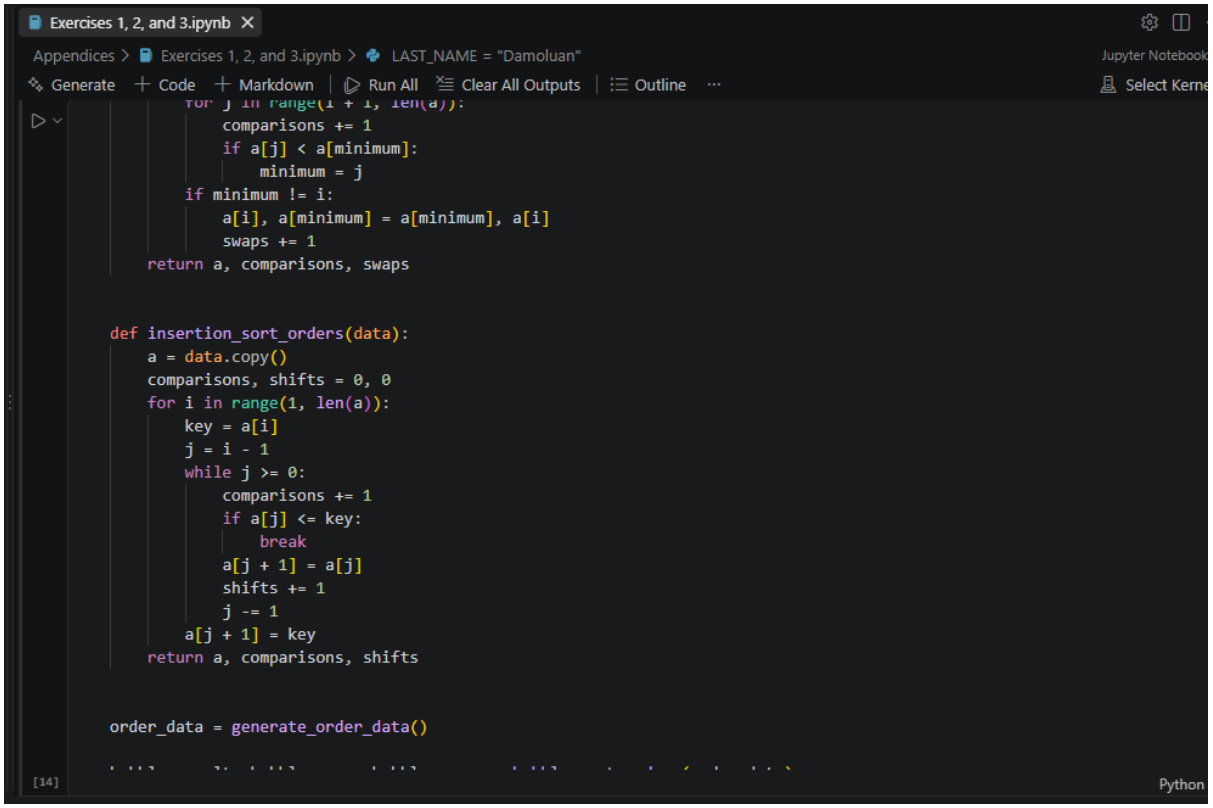
        return dept_a, dept_b

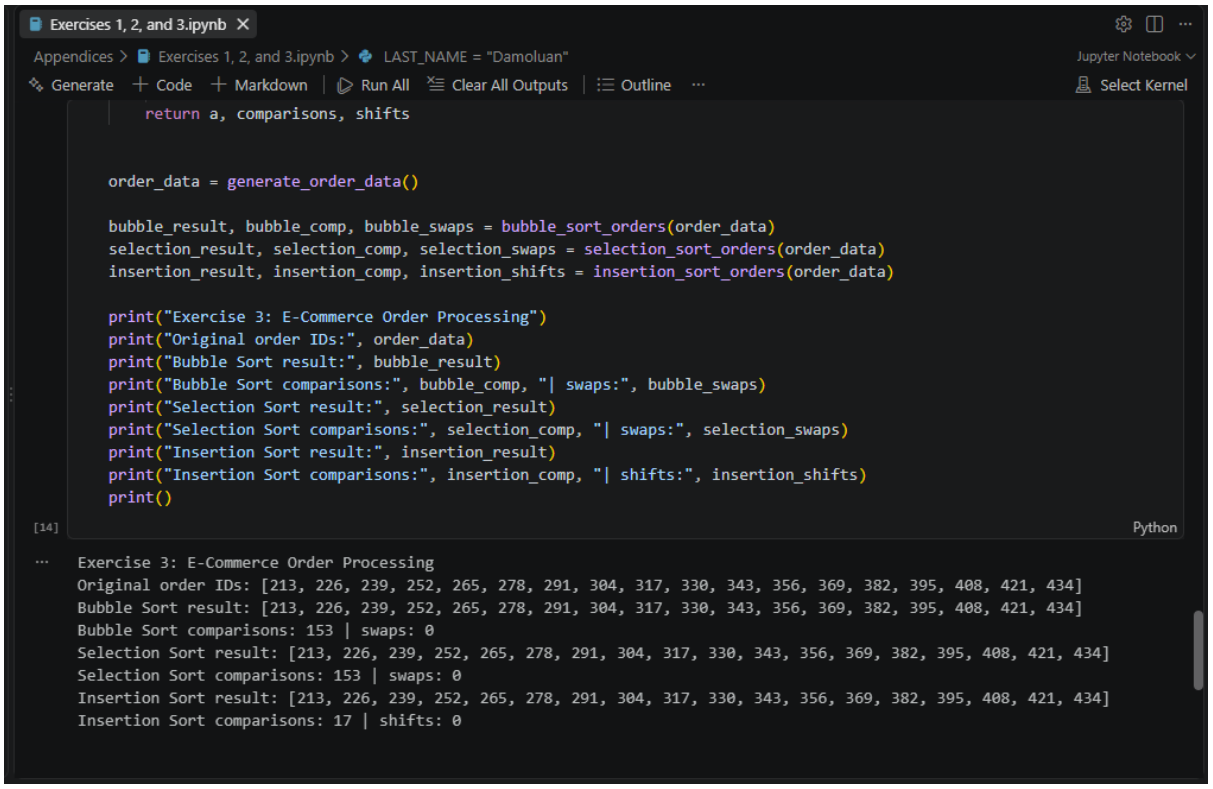

    def consolidate_records(data_a, data_b):
        merged = sorted(set(data_a) | set(data_b))
```

[12] Python


Exercises 1, 2, and 3.ipynb
Appendices > Exercises 1, 2, and 3.ipynb > LAST_NAME = "Damoluan"
Jupyter Notebook
Generate
Code
Markdown
Run All
Clear All Outputs
Outline
Select Kernel
return dept_a, dept_b
def consolidate_records(data_a, data_b):
merged = sorted(set(data_a) | set(data_b))
common = sorted(set(data_a) & set(data_b))
return merged, common
dept_a, dept_b = generate_university_data()
consolidated, common_records = consolidate_records(dept_a, dept_b)
print("Exercise 2: University Enrollment Records")
print("Department A:", dept_a)
print("Department B:", dept_b)
print("Consolidated dataset:", consolidated)
print("Shared records:", common_records)
print("Total records after consolidation:", len(consolidated))
print()
[12]
Python
Exercise 2: University Enrollment Records
Department A: [108, 115, 122, 129, 136, 143, 150, 157, 164, 171]
Department B: [108, 115, 122, 175, 180, 185, 190, 195, 200, 205, 210]
Consolidated dataset: [108, 115, 122, 129, 136, 143, 150, 157, 164, 171, 175, 180, 185, 190, 195, 200, 205, 210]
Shared records: [108, 115, 122]
Total records after consolidation: 18


Exercises 1, 2, and 3.ipynb
Appendices > Exercises 1, 2, and 3.ipynb > LAST_NAME = "Damoluan"
Jupyter Notebook
Generate
Code
Markdown
Run All
Clear All Outputs
Outline
Select Kernel
Appendix C
def generate_order_data():
order_ids = []
for i in range(18):
value = (SEED_DIGIT * 11 + i * 13 + ID_SUM) % 1000
order_ids.append(value + 100)
return order_ids
def bubble_sort_orders(data):
a = data.copy()
comparisons, swaps = 0, 0
for i in range(len(a)):
for j in range(0, len(a) - i - 1):
comparisons += 1
if a[j] > a[j + 1]:
a[j], a[j + 1] = a[j + 1], a[j]
swaps += 1
return a, comparisons, swaps
def selection_sort_orders(data):
a = data.copy()
comparisons, swaps = 0, 0
for i in range(len(a) - 1):
minimum = i
for j in range(i + 1, len(a)):
[14]
Python


Exercises 1, 2, and 3.ipynb
Appendices > Exercises 1, 2, and 3.ipynb > LAST_NAME = "Damoluan"
Jupyter Notebook
Generate
Code
Markdown
Run All
Clear All Outputs
Outline
Select Kerne
for j in range(i + 1, len(a)):
comparisons += 1
if a[j] < a[minimum]:
minimum = j
if minimum != i:
a[i], a[minimum] = a[minimum], a[i]
swaps += 1
return a, comparisons, swaps
def insertion_sort_orders(data):
a = data.copy()
comparisons, shifts = 0, 0
for i in range(1, len(a)):
key = a[i]
j = i - 1
while j >= 0:
comparisons += 1
if a[j] <= key:
break
a[j + 1] = a[j]
shifts += 1
j -= 1
a[j + 1] = key
return a, comparisons, shifts
order_data = generate_order_data()
[14]
Python


Exercises 1, 2, and 3.ipynb
Appendices > Exercises 1, 2, and 3.ipynb > LAST_NAME = "Damoluan"
Jupyter Notebook
Generate
Code
Markdown
Run All
Clear All Outputs
Outline
Select Kernel
return a, comparisons, shifts
order_data = generate_order_data()
bubble_result, bubble_comp, bubble_swaps = bubble_sort_orders(order_data)
selection_result, selection_comp, selection_swaps = selection_sort_orders(order_data)
insertion_result, insertion_comp, insertion_shifts = insertion_sort_orders(order_data)
print("Exercise 3: E-Commerce Order Processing")
print("Original order IDs:", order_data)
print("Bubble Sort result:", bubble_result)
print("Bubble Sort comparisons:", bubble_comp, "| swaps:", bubble_swaps)
print("Selection Sort result:", selection_result)
print("Selection Sort comparisons:", selection_comp, "| swaps:", selection_swaps)
print("Insertion Sort result:", insertion_result)
print("Insertion Sort comparisons:", insertion_comp, "| shifts:", insertion_shifts)
print()
[14]
Python
Exercise 3: E-Commerce Order Processing
Original order IDs: [213, 226, 239, 252, 265, 278, 291, 304, 317, 330, 343, 356, 369, 382, 395, 408, 421, 434]
Bubble Sort result: [213, 226, 239, 252, 265, 278, 291, 304, 317, 330, 343, 356, 369, 382, 395, 408, 421, 434]
Bubble Sort comparisons: 153 | swaps: 0
Selection Sort result: [213, 226, 239, 252, 265, 278, 291, 304, 317, 330, 343, 356, 369, 382, 395, 408, 421, 434]
Selection Sort comparisons: 153 | swaps: 0
Insertion Sort result: [213, 226, 239, 252, 265, 278, 291, 304, 317, 330, 343, 356, 369, 382, 395, 408, 421, 434]
Insertion Sort comparisons: 17 | shifts: 0